Sketchbook: Space-Efficient and Workload-Independent Frequency Estimation

Navid Eslami
navideslami@cs.toronto.edu
University of Toronto
Toronto, Canada

Ioana O. Bercea
bercea@kth.se
KTH Royal Institute of Technology
Stockholm, Sweden

Rasmus Pagh
pagh@di.ku.dk
University of Copenhagen
Copenhagen, Denmark

Niv Dayan
nivdayan@cs.toronto.edu
University of Toronto
Toronto, Canada

ABSTRACT

PVLDB Reference Format:
Navid Eslami, Ioana O. Bercea, Rasmus Pagh, and Niv Dayan. Sketchbook: Space-Efficient and Workload-Independent Frequency Estimation. PVLDB, 18(1): XXX-XXX, 2025.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/n3slami/Sketchbook.git.

1 INTRODUCTION

The streams we consider both insert and delete keys, which amount to incrementing and decrementing the key's count, respectively. While some applications such as tracking click-through rates in webpages only require insertions, deletions are needed for other applications such as maintaining count estimates for SQL tables for use in a query optimizer.

Rasmus: For deletions it is critical to note that they are not "checked for validity" by the sketches, so it is up to the user to ensure that deletions correspond to insertions

We define the following two goals: **(G1) Space-Efficient Counters**, and **(G2) Scaling Error with Stream Length**.

Applications. Due to its simplicity and high accuracy under skewed workloads, CMS is used in many applications, including k-mer counting in bioinformatics [49, 60], network monitoring [14, 30, 34], and hardware security systems [5]. Yet, its biased estimates make it less useful for applications such as join size estimation. As shown in Section 4.3, the Count Sketch is the tool of choice for these settings.

Since CS provides unbiased estimates, it is used in applications where estimates are combined or multiplied to derive more complex statistics. In particular, unbiased estimation is crucial for query optimization [25, 27, 28] and covariance estimation [16], as count estimates are multiplied in these contexts, amplifying any bias. Other applications benefiting from CS include feature selection [1] and similarity search [24, 29, 52, 53]. CS also provides *Universality* [6]: the ability to estimate many statistics based on the key frequencies. This makes it a general measurement tool for networks since it allows administrators to collect diagnostics of any kind without needing to explicitly define the required statistics (such as the mean, variance, and moments) beforehand [38].

As MG accurately tracks the frequencies of heavy hitters, many other sketch designs use it as a sub-component to reap its benefits. Moreover, because it also stores the keys themselves, unlike CMS and CS, it is used to report the heavy hitters, trending keys, and much more [3, 7, 26, 33, 36, 37, 47, 48, 50, 54, 55, 57, 59].

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 18, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

2 BACKGROUND

A stream is a sequence of key-value pairs $\langle (x_1, \Delta_1), (x_2, \Delta_2), \dots \rangle$, where x_i is the i-th key that appears in the stream and $\Delta_i = \pm 1$ represents whether its count was incremented due to an insertion ($\Delta_i = 1$) or decremented due to a deletion ($\Delta_i = -1$). This is known as the *General Turnstile Model* of data streams [23]. We denote key x's count by $f(x)$, and represent the current total count of the keys in the stream with $N = \sum_x f(x) = \sum_i \Delta_i$. Table 1 lists the terms and symbols used throughout the paper.

Exhaustive Approach. The simplest approach to tracking frequencies is to store each key and its count in a hash table. Although this approach is fully accurate, it entails a high memory overhead for three reasons: (i) It stores all keys in the stream. Variable-length keys worsen this overhead since they require an indirection layer (e.g., pointers). (ii) It stores one counter for each unique key. (iii) The hash table may occasionally be as much as half empty to support collision resolution and expansions. These different overheads multiply with the number of streams the application monitors. Problems (i) and (ii) imply a memory footprint that is linear in the number of unique keys in the stream, which can be of the same magnitude as the total key count N.

Frequency estimation sketches address these problems by trading accuracy for space. That is, for any key x, a sketch returns an estimate $\hat{f}(x)$ of x's count $f(x)$. We focus on Count-Min Sketch [13] for now, as it is the simplest and most widely used sketch [35, 40]. Later, we describe Count Sketch [8] (Section 4.3) and Misra-Gries [43] (Appendix A) and apply our techniques to them.

Count-Min Sketch (CMS). The core design element of CMS is a counter array consisting of w counters, each initialized to zero.

Table 1: Definitions of terms and symbols.

Symbol	Definition
N	The sum of the counts of all keys in the stream.
x, y	Example keys from the stream.
$f(x)$	The ground-truth count of key x.
$\hat{f}(x)$	The estimated count of key x.
w	Number of counters/slots.
d	Number of independent arrays.
$h_i(x)$	Hash function mapping key x to a counter.
$\sigma_i(x)$	Hash function defining key x's update direction (±1).
$N^{\text{res}(i)}$	Total count of all but the i most frequent keys.
c	Number of counters in each chunk.
s	Length of each stub in bits.
$W(N)$	Size function.

We insert a key x into this array by hashing it to one counter and incrementing it. We delete x by decrementing its corresponding counter.[1] The counter that key x hashes to provides an estimate of x's count. Due to hash collisions, multiple keys can map to the same counter. Because of this, we may overestimate the count of a key x. This error is at most $\frac{N}{w}$ in expectation since a key participates in a hash collision with a probability of $\frac{1}{w}$ and the total count of keys different from x is at most N. Increasing the array's size makes collisions less likely, thereby decreasing the error and improving *Accuracy*. Even so, the error may deviate from its expectation. The probability that errors do not exceed the expectation "too much" is known as the sketch's *Confidence*. A straight-forward application of Markov's inequality shows that the error does not exceed $\frac{eN}{w}$ with a confidence of at least $1-e^{-1}$, i.e., $\Pr\left(|\hat{f}(x)-f(x)|\leq \frac{eN}{w}\right) \geq 1-e^{-1}$ for any key x. Here, we use Euler's constant e due to convention. Any other constant would also work and would replace e in both the accuracy bound and the confidence. This implies that the smaller the constant becomes, i.e., the more accurate we expect the sketch to be, the lower confidence drops.

Yet, with a single array, confidence is low (≈ 63%). The reason is that all keys sharing the same counter as a frequent key will suffer from inflated estimates. Moreover, since there are many keys with low or moderate counts, there is a non-negligible chance that enough of them map to the same counter and inflate estimates beyond the error bound. CMS addresses these issues by allocating d independent counter arrays, each with a dedicated hash function h_i. CMS inserts or deletes a key by applying the operation to each array, and it answers a query by querying all arrays and taking the minimum of the estimates.[2] Figure 1 shows an example of insertions and queries applied to three counter arrays. Returning the minimum for queries boosts confidence exponentially, since it only carries severe errors when all estimates err significantly. Thus, CMS guarantees an error of at most $\frac{eN}{w}$ with a confidence of $1-e^{-d}$, i.e., $\Pr\left(|\hat{f}(x)-f(x)|\leq \frac{eN}{w}\right) \geq 1-e^{-d}$ for any key x.

[1]Users must ensure deletions target previously inserted keys. This is because frequency estimation sketches do not store the keys and cannot check a deletion's validity.
[2]CMS uses pairwise-independent hash functions to bound the probability of collisions and overestimation [13].

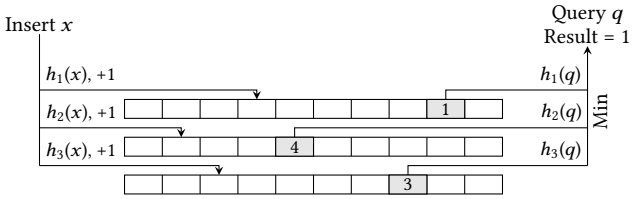


Figure 1: CMS maintains d counter arrays of size w and hashes keys into them to estimate their frequencies. Insertions are illustrated on the left and queries on the right.

Since CMS uses a constant number of tightly packed counters and does not store the stream's keys, it significantly improves the memory footprint compared to the exhaustive approach of tracking each key's exact count. Nevertheless, CMS still suffers from several memory inefficiencies, as we show in the following section.

3 PROBLEM ANALYSIS

Traditional frequency estimation sketches, such as CMS, use more memory than necessary by failing to account for 1) the skew in the stream and 2) the stream's gradual growth over time.

Challenge 1: Skew. With uniform workloads, most counters in CMS take on similar values. In contrast, under skewed workloads with a few heavy hitters, most counter values remain small while only a few grow large. Because CMS relies on uniformly sized counters that must be sufficiently long to store the maximum counter value, it wastes substantial memory by allocating excessive bits to the many counters with small values. Ideally, each counter should be just long enough to fit its value, thereby saving space or allowing more counters to improve accuracy.

A common strategy for tackling this problem is to track the heavy hitters separately in a hash table while leaving the rest of the keys to CMS [33, 47, 50, 55–57]. Such hybrid structures aim to cap the maximum counter value in CMS, thereby permitting shorter counters. Yet, this approach does not fundamentally resolve the issue. The keys tracked by CMS may still follow a skewed distribution, leading to significant memory wastage. Moreover, the heavy hitters can vary over time. Identifying the next heavy hitter and "promoting" it from CMS into the hash table is error-prone, since collisions in CMS can cause many infrequent keys to be misclassified as heavy hitters. The hybrid structures in the literature explore different hash table implementations and promotion policies.

Other approaches compress the counters using algorithms such as compressive sensing [21, 39] or hypergraph peeling [10]. Yet, these methods impose heavy decompression overheads on queries.

Counter sharing approaches store the counter values in a hierarchy of counters, with the lowest level storing the counter values in short counters and the upper levels handling overflows using shared counters [9, 21, 22, 32, 39, 58]. The left side of Figure 2 encodes four adjacent counters within CMS using such a hierarchy with 6-bit and 2-bit counters. Whenever a counter in this hierarchy overflows, it increments its parent and resets itself back to zero. A counter's value is retrieved by concatenating the counters along its path to the root until reaching a delimiter, typically indicated using additional metadata. While counter sharing yields state-of-the-art

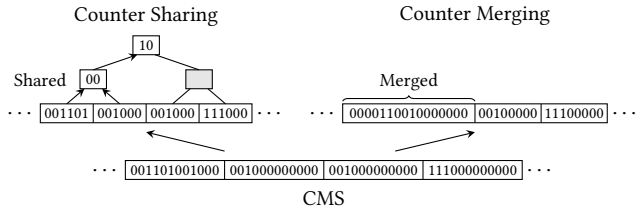


Figure 2: Counter sharing and counter merging encode CMS's counters in less space in exchange for blowing up some counter values. Here, the bits in the binary representations are in increasing order of significance from left to right.

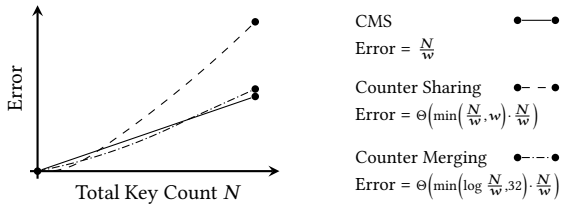


Figure 3: Under the same memory budget as a CMS instance with 32-bit counters, counter sharing and counter merging can lead to lower accuracy when processing a growing stream.

accuracy and performance for short streams, its error rapidly deteriorates as the stream grows. In particular, after N insertions into an array of size w, each counter overflows into the $\Theta(\log(N/w))$ levels above it. This blows up the values of $\Theta(N/w)$ other counters, increasing the estimation error by the same factor. The counter sharing methods in the literature experiment with the hierarchy's structure, including aspects like the counter sizes and the fanout.

Counter merging is an alternative approach that starts with 8-bit counters and merges adjacent overflowing counters into a longer counter that sums up the original values [4]. The right side of Figure 2 applies this scheme to CMS. After N insertions into an array of w counters, each counter will be merged with $\Theta(\log(N/w))$ other counters, increasing the error by the same factor. Merging only continues until the counters become the same length as those of CMS, at which point error degradation stops. Counter merging has a higher metadata overhead than counter sharing for indicating which counters are merged, leaving less space for storing counters. Parsing this metadata is also complex, reducing performance.

Deletions deteriorate the accuracy of both counter sharing and counter merging methods since these techniques ambiguate how counters should be "separated" as they decrease.

Figure 3 qualitatively compares counter sharing and counter merging to CMS to illustrate their error degradation with data growth, under the same memory budget. While these methods perform well for short streams, their error exceeds that of CMS as the stream grows. We empirically verify this behavior in Section 6.

Challenge 2: Unbounded Growth. In many streaming applications, the data stream grows continuously, and it is imperative to estimate the count of each key from the stream's beginning. For instance, query optimizers often track the number of times each key occurs in a table column to estimate the size of a join on that column [25, 27, 28]. As new data is inserted, the total number of keys N increases. Yet, the full count history of each key is still needed, since every occurrence contributes equally to the join size regardless of when it arrived.

Unbounded stream growth is also a challenge for applications that only require key counts over recent time windows. For example, network management systems detect anomalous or malicious activity by counting packets from each source within fixed time windows [14, 30, 34]. With bursty streams such as network traffic, the number of keys N in each window is unpredictable.

In existing frequency estimation sketches, the number of counters w is fixed at allocation time. Consequently, the error rate degrades linearly with N (e.g., N/w in the case of CMS). This forces practitioners to tune w without knowledge of the final stream length, hoping to balance accuracy and the memory footprint. Having too few counters leads to significant errors, while having too many wastes memory. Deletions make the problem more challenging by causing N to fluctuate over time. Worse yet, even if the ultimate stream length is known upfront, allocating a large CMS from the start wastes memory during the early stages when the stream is short.

Summary. An ideal frequency estimation sketch should: 1) compactly encode counter values without incurring memory waste from skew, 2) scale its size with the stream's length to keep both the memory footprint and the error rate under control, 3) support deletions without sacrificing accuracy, and 4) provide high-performance operations. Is it possible to achieve all these goals simultaneously?

4 SKETCHBOOK

We present Sketchbook, a framework for generalizing a frequency estimation sketch to optimize its memory footprint with respect to the stream's skew and its length. Section 4.1 shows how to adapt to skew by compactly encoding each counter as a short integer and extending it as it overflows. Section 4.2 shows how to control both accuracy and space for a stream of unknown length by expanding and contracting the sketch. At the same time, Sketchbook supports deletions and maintains high performance. We first describe Sketchbook as applied to the Count-Min Sketch (CMS), denoted as $\text{Sketchbook}_{\text{CMS}}$. We apply it to other sketches in Section 4.3 and Appendix A.

4.1 Accommodating Skew

Under skew, many counter values in CMS will be small and not use all the bits in their counters. To optimize the space used for storing these counter values in any workload, $\text{Sketchbook}_{\text{CMS}}$ generalizes CMS with a general-purpose variable-length counter (VALE) encoding. VALE stores the counters as fixed-size integers called *Stubs*. When a counter overflows its stub, VALE elongates it with a dedicated variable-length *Extension*. Due to their need for delimiters, storing extensions entails a slight metadata overhead. We counteract this overhead by choosing the stub length based on the workload's skew so that more counter values fit in their stubs without needing extensions. More precisely, we choose the shortest stub length that minimizes the total memory footprint of the

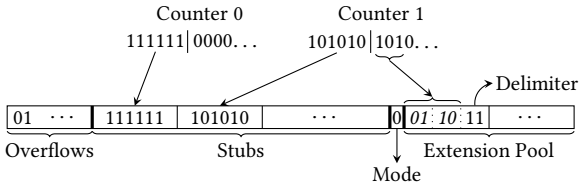


Figure 4: VALE encodes a chunk of c counters in a cache line. It encodes the s lower-order bits of each counter in a stub and stores its remaining higher-order bits in extensions comprised of 2-bit fragments representing base-3 digits. Here, we use a stub length of $s = 6$.

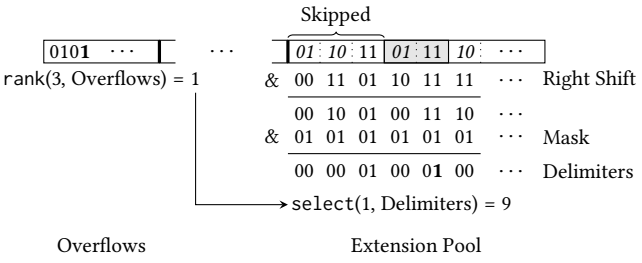


Figure 5: Sketchbook$_{\text{CMS}}$ applies rank and select operations to locate an overflowing counter's extension.

stubs and the extensions. We adapt this stub length to the workload by reconfiguring it when the total number of overflows exceeds a threshold. To improve cache behavior and quickly reconstruct a counter, VALE colocates a stub and its corresponding extension in the same cache line. It quickly finds a stub's extension using a bit manipulation workflow that leverages rank and select operations.

Chunking. To colocate stubs and their extensions in 512-bit cache lines, VALE partitions each counter array into contiguous *Chunks*, each occupying a cache line. A chunk stores its counters' stubs in a contiguous array and tracks the counters that overflow their stubs using an *Overflows Bitmap* with one bit per counter. It stores the extensions of these overflowing counters contiguously in the remaining space towards its end, called its *Extension Pool*. Counter 0 and Counter 1 in Figure 4 are examples of a non-overflowing and overflowing counter within a chunk, respectively.

Parameters. VALE has two parameters: the number of bits in each stub s and the number of counters per chunk c. With more skewed workloads, having shorter stubs and more counters per chunk reduces space consumption. The reason is that skew shrinks most counter values, enabling the use of shorter stubs for storing them. Skew also reduces the average length of the extensions, leaving more space in a chunk to store more counters.

Later in this section, we discuss how to adapt the tuning of these parameters to the workload as the stream evolves. When the workload is uniform, this procedure tunes the stub length so that all counter values fit in stubs, allowing it to remove the overflows bitmaps and yield the configuration of a standard CMS. We illustrate the impact of this procedure on the memory footprint in Section 6.

Extensions. We form extensions from 2-bit *Fragments* to encode the higher-order bits of overflowing counters succinctly and to extend them at the finest possible granularity. We use the fragment 11 as a delimiter at the end of each extension. The other three combinations of bits are used to encode each extension's value in base 3. Specifically, the base-3 digits $(0)_3$, $(1)_3$, and $(2)_3$ are represented as the fragments *00*, *10*, and *01*, respectively. We store fragments in increasing order of their digits' significance from left to right. Formally, a value v is encoded in an extension by representing each of its $\lfloor \log_3 v \rfloor + 1$ digits in a fragment, with the j-th digit being $v_j = \lfloor v/3^j \rfloor \mod 3$. We decode v back into decimal by computing the sum $3^0 \cdot v_0 + 3^1 \cdot v_1 + \ldots$.

For example, the value 5, which is the value of the higher-order bits of Counter 1 in Figure 4, has two digits in base 3. It is therefore encoded as the extension $\langle (2)_3, (1)_3, \text{delimiter} \rangle = \langle 01, 10, 11 \rangle$. Decoding this extension yields $3^0 \cdot 2 + 3^1 \cdot 1 = 5$, as expected.

Forming a chunk's extensions from 2-bit fragments allows them to fit in 1-2 machine words in most cases. We prove in Section 5 that using these extensions in conjunction with short stubs enables VALE to encode each counter's value in space close to the length of its binary encoding.

Treating arbitrary variable-length data as counters allows VALE to compactly encode them. Thus, it can be used in other applications operating on such data to improve memory efficiency.

Locating an Extension. To decode an overflowing counter, VALE must locate its extension. To this end, it keeps the extensions in the same order as that of the counters. It locates the i-th counter's extension by counting the number of preceding bits set to 1s in the overflows bitmap. This is equivalent to the number of overflowing counters preceding the i-th counter, denoted by m. Skipping over the first m extensions in the extension pool leads to the position of the target extension.

Figure 5 illustrates the process of identifying the 3rd counter's extension in a chunk. Here, since the 3rd bit in the overflows bitmap is 1, the 3rd counter has an extension. As there is one bit set to 1 preceding the 3rd bit in the overflows bitmap, we skip one extension to locate the 3rd counter's extension, shown in gray.

Fast Access Using Rank and Select. VALE quickly locates the extension of an overflowing counter using rank and select operations [11, 31, 44, 46, 61]. Given a bitmap B, the $\texttt{rank}(i, B)$ operation counts the number of 1s strictly before the i-th bit, and the select(i, B) operation returns the position of the i-th 1. Later in this section, we show how to use specialized CPU instructions to implement these operations efficiently.

The first step is to determine m, the number of overflowing counters to the left of the i-th counter. As this is the number of 1s before the i-th bit in the overflows bitmap, VALE applies the rank operation to evaluate $m = \texttt{rank}(i, \text{overflows})$.

The second step is to identify the target extension's delimiter by skipping over the m delimiters preceding it. We do this using several bit manipulation techniques. First, we convert the extension pool into a so-called *Delimiters Bitmap*. This bitmap contains one bit set to 1 at the position of every delimiter and 0s everywhere else, as illustrated at the bottom right of Figure 5. To derive this bitmap, we take the bitwise-and of the extension pool with a version of itself shifted to the right by one bit. The result is a bitmap with

a 1 at the end of any two consecutive 1s in the original extension pool, as illustrated in Figure 5. This means that every bit in this bitmap corresponding to the second bit of a fragment is set to 1 if the fragment is a delimiter and is set to 0 otherwise. We mask out all other bits to 0s. In sum, the delimiters bitmap is derived as

$$\text{delimiters} = \text{extensions} \;\&\; (\text{extensions} >> 1) \;\&\; 0101\ldots0101.$$

Finally, we use the select operation to evaluate `select`(m, delimiters), which skips over the first m delimiters and returns the position of the m-th delimiter. We sequentially apply the bitwise operations above to each machine word comprising the chunk's extensions. This is efficient, as a chunk's extensions often fit in 1-2 words.

Figure 5 shows an example of locating the 3rd counter's extension using rank and select. Here, `rank`(3, overflows) = 1. Thus, after deriving the delimiters bitmap by manipulating the extension pool, VALE evaluates `select`(1, delimiters) = 9 to conclude that the target extension ends at the 9th bit.

Tails. In the rare event that many heavy hitters hash to the same chunk, many of the chunk's counters will overflow, potentially growing the cumulative length of the extensions beyond the size of the chunk. VALE handles this case by allocating an external array of 32-bit integers called *Tails* for the chunk, with one tail integer per counter. It stores the i-th counter's higher-order bits in the i-th tail. Each chunk contains a mode flag indicating if it has a tails array, as depicted in Figure 6. If it does, we repurpose the space taken up by the extension pool to store a pointer to the tails array. We ensure that the extension pool is large enough to accommodate this pointer (i.e., at least 48 bits[3]) when tuning the number of counters per chunk and the stub size. Any space after the pointer is unused.

Adaptive Tuning. If more than 1% of the chunks use tails arrays after an insertion or if there are more than two unused bits per counter after a deletion, Sketchbook$_{\text{CMS}}$ adapts VALE's tuning, i.e., the number of counters per chunk c and the stub length s, to minimize the memory footprint. It does this by scanning the entire sketch and computing the number of counter values of every binary length. It uses this summary to calculate the expected total length of a chunk's extensions for a given combination of parameters c and s. It enumerates the possible combinations by first varying c from largest to smallest and enumerating the stub length s from smallest to largest for each fixed c. For each combination, it then applies Chebyshev's inequality to the total extension length to derive a conservative bound on the number of chunks that will use tails arrays. It chooses the first combination that yields a bound smaller than 0.5%. In doing so, Sketchbook$_{\text{CMS}}$ ensures VALE packs as many counters as possible within each chunk and the stubs have as few unused bits as possible. It also ensures that extensions rarely outgrow their chunks, making the memory overhead of the tails arrays negligible. The amortized overhead of this tuning process is constant, as many more insertions and deletions compared to the number counters must take place before this process is triggered.

Incrementing Counters. When an insertion maps to the i-th counter of a chunk, Sketchbook$_{\text{CMS}}$ increments it by adding one to the lower-order bits stored in the i-th stub, i.e., stub[i]. Most of the time, stub[i]'s increment will not carry over to the higher-order bits

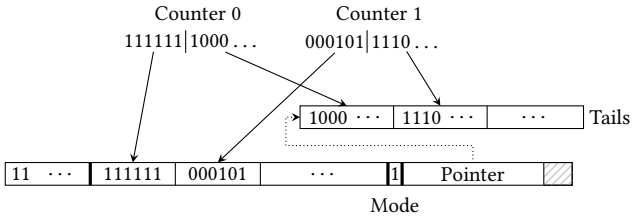


Figure 6: When a chunk's extensions outgrow its space, it stores a pointer to a dedicated external array of tails to store the counters' higher-order bits.

of the counter, allowing the insertion to return early and achieve high throughput.

Whenever stub[i]'s increment carries over, Sketchbook$_{\text{CMS}}$ adds one to the counter's extension or tail. For an extension, this entails incrementing its first digit and propagating a carry to the other digits as necessary. This carry may propagate beyond the end of the extension, leading to the addition of a new digit. Sketchbook$_{\text{CMS}}$ makes room for the fragment representing this digit by shifting subsequent extensions to the right. If the i-th counter did not have an extension, a new extension encoding the value 1 is created for it. We use rank and select operations to determine the appropriate position of this extension in the extension pool.

Decrementing Counters. Analogously, when a deletion maps to the i-th counter in a chunk, Sketchbook$_{\text{CMS}}$ decrements the lower-order bits in stub[i]. If stub[i] borrows from the counter's higher-order bits as a result, Sketchbook$_{\text{CMS}}$ decrements the corresponding extension or tail. This may trigger the removal of a digit from the counter's extension, in which case subsequent extensions are shifted to the left to keep the extension pool tightly packed.

Optimization 1: Fast Division. Determining the chunk of a counter at offset i in an array is expensive, as it entails dividing i by the number of counters per chunk c, which may not be a power of two. We avoid this cost by simulating the division through fixed-point multiplication. Specifically, we compute the expression $\left\lfloor \left(i/2^{\lceil \log_2 c \rceil}\right) \cdot \left(2^{\lceil \log_2 c \rceil}/c\right) \right\rfloor = \lfloor i/c \rfloor$ by representing the multiplicands as fixed-point fractional numbers. To this end, we precompute the latter multiplicand's fixed-point representation with $\lceil \log_2 w \rceil$ bits of precision, where w is the size of an array. For the former multiplicand, we treat i's binary representation as a fixed-point fractional value with the decimal point being after the $\lceil \log_2 c \rceil$-th bit. Finally, we multiply these values using standard integer multiplication and shift the result $\lceil \log_2 w \rceil + \lceil \log_2 c \rceil$ bits to the right to remove the fractional bits and compute the answer.

Optimization 2: Hardware Accelerated Rank and Select. Sketchbook$_{\text{CMS}}$ utilizes specialized CPU instructions to implement rank and select operations. It implements `rank`(i, B) by applying the `POPCNT` instruction to the first $\lceil i/64 \rceil$ words of the bitmap B to count the 1s before the i-th bit.[4] It also implements `select`(i, B) by counting the 1s in B's words to locate the word containing the i-th 1. Once found, it applies the x86 instructions `PDEP` and `TZCNT` to evaluate the target bit's position within that word, as described

[3]The upper 16 bits a general 64-bit pointer are unused on modern machines. As such, we only store the lower 48 bits of the pointer.

[4]Formally, denoting the j-th word in B as B_j, this process corresponds to computing $\texttt{POPCNT}\left(B_{\lfloor i/64 \rfloor} \;\&\; ((1 << (i \mod 64)) - 1)\right) + \sum_{j=0}^{\lfloor i/64 \rfloor - 1} \texttt{POPCNT}(B_j)$.

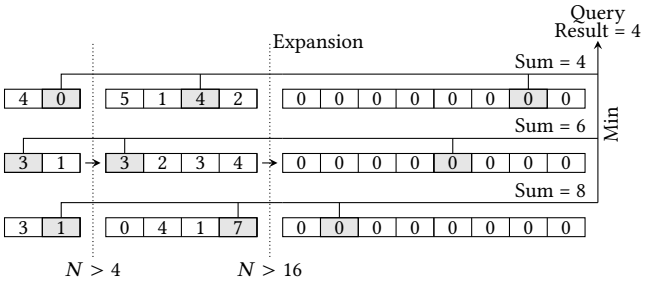


Figure 7: Sketchbook$_{CMS}$ maintains a chain of sketches with exponentially larger arrays.

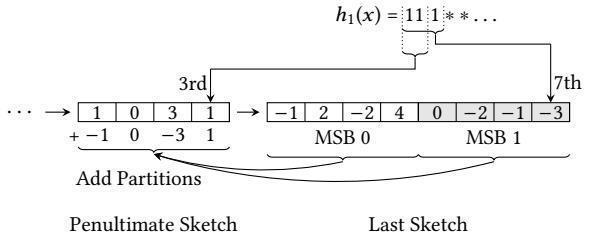


Figure 8: Sketchbook$_{CMS}$ uses fixed hash functions for all sketches in the chain to enable transferring information from the last sketch during contractions.

in [46].[5] Each of these commands is applied to 1-2 machine words, as the bitmaps Sketchbook$_{CMS}$ operates over are 1-2 words long.

Optimization 3: Lookup Tables. Since stubs are not byte-aligned, modifying them can be costly. This is because extracting and writing back a stub to the machine words containing it entails five extra bitwise operations on top of the addition operation. Sketchbook$_{CMS}$ bypasses this cost by directly updating the stub's words using a single addition. It achieves this using two precomputed lookup tables. The first table stores the position of the word each stub lies in, and the second table stores the stub's position within its word. Sketchbook$_{CMS}$ updates a stub by adding or subtracting one from the bit indicated by the second table in the word indicated by the first. For example, in the context of Figure 4, the first lookup table indicates that the 1st stub lies in the 1st word, which has a binary representation of 111111101010 . . . . The second lookup table indicates that this stub starts at the 6th bit in the 1st word, allowing us to increment it by adding 000000100000 . . . , thus yielding 111111011010 . . . as the updated word.

Optimization 4: Prefetching. We allow multiple chunks to be read in parallel within a single operation as well as across multiple operations using prefetching techniques similar to that of [32]. Specifically, we maintain a prefetching queue for each array in Sketchbook$_{CMS}$ that records the eight most recent requests for chunks. Each time a chunk is requested from an array, a software prefetch instruction is issued for it, and the request is pushed into the array's queue. When a queue fills up, the chunk of its oldest request will be present in the L1 cache, allowing it to be read efficiently while other chunks are being loaded into the cache.

Optimization 5: Hash Splicing. Recall that a key is hashed into each of the sketch's d arrays during an update or query. Instead of applying d separate hash functions and incurring their CPU overheads, Sketchbook$_{CMS}$ applies a single hash function to the key to compute a long hash. It splices the result into d sub-hashes of equal length and uses each as the hash value of an array.

4.2 Accommodating Unbounded Growth

Recall from Section 3 that processing growing streams with existing frequency estimation sketches (e.g., CMS) yields an estimation error that deteriorates linearly with the total key count N. Sketchbook addresses this issue for each sketch by generalizing it to expand the number of its counters as the stream grows. In the case of CMS, Sketchbook$_{CMS}$ first allocates a sketch with arrays consisting of a few counters. It inserts and deletes keys from this sketch until N exceeds an internally chosen threshold. Once this occurs, Sketchbook$_{CMS}$ expands by allocating a new sketch with arrays twice the size of those in the old sketch, with counters initialized to zeros. It directs insertions and deletions to this new sketch so that they benefit from its higher tracking accuracy. Whenever N again exceeds a threshold, Sketchbook$_{CMS}$ allocates another sketch double the size of the previous one. It arranges these sketches from oldest to newest in a chain, yielding the structure shown in Figure 7. All sketches along this chain have the same number of arrays d to enable a constant confidence bound. A query checks each sketch in the chain, sums the estimates from corresponding arrays, and reports the minimum.

Unlike arrays or hash tables, which expand when N doubles, Sketchbook$_{CMS}$ expands after N more than doubles to achieve both sub-linear errors and a sub-linear memory footprint. More precisely, it chooses the expansion thresholds as the values of N that double the value of a user-defined *Size Function*, denoted as $W(N)$. We set $W(N) = \sqrt{N}$ by default, which corresponds to expanding the sketch whenever N quadruples, as shown at the bottom of Figure 7. These thresholds keep the size of the last sketch's arrays within a factor of two of $W(N) = \sqrt{N}$. Moreover, they cause the last sketch to process the majority of the stream, making it the dominant factor in both the overall error bound and memory. Intuitively, this means that Sketchbook$_{CMS}$ simulates processing the entire stream with a sketch allocated upfront with arrays of size $\sqrt{N}$, as if N was known.

Contractions. When deletions decrease N below the previous expansion threshold, i.e., whenever N drops by 4×, Sketchbook$_{CMS}$ contracts to save memory. It does so by transferring the updates of the last sketch to the penultimate sketch and discarding the last sketch. This transfer is challenging, as we do not have access to the keys and thus cannot rehash them into the penultimate sketch. We overcome this issue by using d shared hash functions $h_1, \ldots, h_d$ for the sketches, each computing a long *Mother Hash* [18, 19, 45] for a given key. To map a key x to a counter in the i-th array of a sketch, we compute $h_i(x)$ to derive its mother hash. Letting w denote the size of this array, we take the $\log_2 w$ lower-order bits of x's mother hash as the offset of the counter it hashes to. Figure 8 shows this procedure for the first arrays of the chain's last two sketches.

[5] If the j-th word B_j contains the i-th 1 and there are r 1s in the preceding words, the result is computed as $64 \cdot j + \texttt{TZCNT}(\texttt{PDEP}((1 << (i - r)) - 1, B_j))$, where the first operand of PDEP is the payload and the second operand is the mask [12].

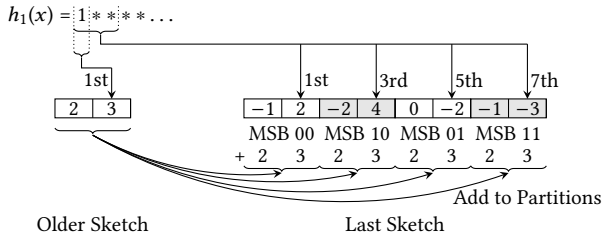


Figure 9: Sketchbook$_{CMS}$ aggregates an older sketch into the last sketch by partitioning the latter based on the MSBs of the counter offsets and adding the former to each partition.

This hashing scheme allows Sketchbook$_{CMS}$ to determine the counter a key hashes to in the penultimate sketch by removing the most significant bit (MSB) of its counter's offset in the last sketch. Thus, adding the value of the latter counter to the former transfers the effects of all updates that hashed to it. Since the MSB value of the counters' offsets partitions the last sketch into two halves, transferring all updates amounts to adding each partition to the penultimate sketch. We illustrate this at the bottom of Figure 8.

Because the majority of the stream's keys must be deleted to trigger a contraction, the last two sketches of the chain always handle the majority of the stream's keys. Therefore, together, they are the dominant factor in both the overall error bound and memory. As such, the error remains bounded, despite the state of each sketch in the chain depending on the sequence of insertions and deletions.

As shown in Figure 8, deletions may turn the counters in the last sketch negative. We shortly show how to avoid such negative counters by aggregating the old sketches into the last one.

Aggregating Sketches. While insertions and deletions only affect the last sketch in the chain, queries must probe all sketches and aggregate their results. Sketchbook$_{CMS}$ speeds up this process by aggregating the updates of all older sketches into the last one without incurring extra errors. It achieves this by leveraging the mother hashes. As before, removing the MSBs of the offset of a key x's counter in the last sketch yields the offset of the counter it hashes to in an older sketch, as shown in Figure 9. By aggregating the value of the latter counter into the former, we ensure that querying x in the last sketch takes into account the old counter's value without needing to probe the old sketch directly. Applying this procedure to all counters in the last sketch amounts to partitioning the last sketch based on the MSBs of its counters' offsets and adding the old sketch to each partition.

Counters in the aggregated sketch cannot be negative. This is because a negative aggregated counter corresponds to the user deleting a key more times than inserting it, which is impossible.

Sketchbook$_{CMS}$ does not lose access to the original last sketch as a result of this aggregation. It reconstructs the last sketch when needed by subtracting the old sketches from the aggregated sketch. As such, contractions proceed as before by transferring the updates of the last sketch into the penultimate sketch.

Once it computes the initial aggregated sketch, Sketchbook$_{CMS}$ expands by concatenating the aggregated sketch with a copy of itself. Thus, it avoids further aggregations and speeds up expansions.

Insert-Only Workloads. Contractions never occur in insert-only streams. Here, the old sketches are redundant, as they only serve to reconstruct the last sketch from the aggregated sketch during contractions. In such cases, Sketchbook$_{CMS}$ discards the old sketches to reduce the memory footprint by at least 50%.

Size Function. Setting the size function $W(N)$ to functions other than $\sqrt{N}$ exposes a rich spectrum of accuracy-memory tradeoffs. The faster the size function grows with respect to the total key count N, the sooner Sketchbook$_{CMS}$ expands, thereby tracking more of the stream's keys more accurately in exchange for a larger memory footprint. We recommend setting the size function to a *Sub-Linear Power* of the form $W(N) = N^{\alpha}/\epsilon$ and tuning the constants $\alpha \in [0, 1)$ and ϵ. Using such functions captures the behavior of expanding when N increases by a constant factor larger than two. We prove in Section 5 that using a sub-linear power yields an error bound of $O(N/W(N)) = O(\epsilon \cdot N^{1-\alpha})$. This matches the error bound of a sketch preallocated with arrays of size $W(N)$, which is optimal [13]. Setting $\alpha = 0.5$ and $\epsilon = 1$ yields the default $W(N) = \sqrt{N}$.

One may hope to achieve $O(\epsilon)$ errors by setting $W(N) = N/\epsilon$. This translates to Sketchbook$_{CMS}$ expanding when N doubles. Yet, we prove in Section 5 that, without knowing the ultimate length of the stream, the best error bound achievable by a sketch that maintains $O(N/\epsilon)$ counters[6] is $O(\epsilon \cdot \log N)$. By setting $W(N) = N/\epsilon$ and applying slight space optimizations, Sketchbook$_{CMS}$ attains this error bound under the same memory budget and is thus optimal.

4.3 Applying Sketchbook to Count Sketch

Count Sketch (CS). CMS, as described in Section 2, has two key shortcomings: (1) its estimations are biased since it only overestimates and (2) its errors compound when it is used in some tasks, particularly those that multiply frequencies. CS addresses these issues by providing unbiased estimates.[7] The first two rows of Table 2 summarizes the properties of CS and compares it to CMS.

The core component of CS is a counter array of size w coupled with a hash function h_i mapping a key to a counter. CS further associates with each key a random "update direction" from the set $\{-1, +1\}$ by hashing it with another hash function σ_i.[8] It inserts a new key x by incrementing or decrementing its counter according to σ_i, as shown in Figure 10. CS deletes x by subtracting $\sigma_i(x)$ from its counter to undo its insertion. Since each key's update direction is random, the expected interference on x's counter from any other key is zero due to the positive and negative possibilities canceling out. Thus, the value of the counter along the update direction is an estimate of x's frequency, which CS returns during a query. Although this estimate may be smaller or larger than the ground-truth frequency, the zero expected interference allows it to match the ground-truth on average, removing bias. Increasing the array's size reduces the likelihood of hash collisions and the average deviation.

[6] More precisely, we show this for any sketch using $O\left(N/\epsilon \cdot \frac{\log\log N}{\log N}\right)$ bits of space.

[7] In CMS, one can subtract the average of the counter values a query key does not hash to from the query's counter to derive an unbiased estimate [20]. Yet, the resulting estimate has a marginally higher variance than that of CS.

[8] The σ_i form a 4-wise independent family of hash functions, yielding a bound on the probability of large errors [8].

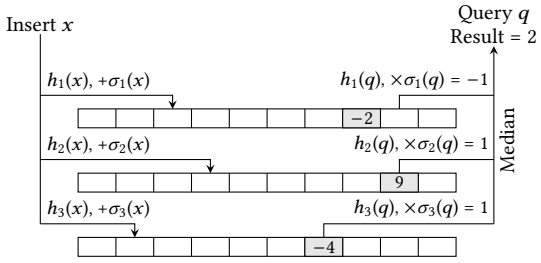


Figure 10: CS assigns a random update direction to each key and uses it to insert them into d counter arrays of size w. The random update directions enable unbiased estimation.

Table 2: The classical sketches strike different tradeoffs.

Sketch	Bias	Best Skew	Accuracy	Confidence	Insert	Delete	Query
CMS	Over	Medium	$\frac{eN}{w}$	$1 - e^{-d}$	$O(d)$	$O(d)$	$O(d)$
CS	Unbiased	Low	$\sqrt{\frac{e \cdot \sum_y (f(y))^2}{w}}$	$1 - e^{-d}$	$O(d)$	$O(d)$	$O(d)$
MG	Under	High	$\min_i \frac{N^{\text{res}(i)}}{w-i+1}$	1	$O(\log w)$	-	$O(1)$

As with CMS, a frequent key will adversely impact the estimates of keys sharing the same counter, degrading confidence. CS addresses this by employing d independent arrays, each with its own hash functions h_i and σ_i. Unlike CMS, CS cannot answer a query by returning the minimum estimate, as it may greatly underestimate the result due to two-sided errors. Instead, it returns the median estimate, as shown in Figure 10. This improves confidence since severe errors are unlikely to occur for enough experiments to affect the median. CS takes the median instead of the mean to improve robustness to outlier estimates. With d arrays with w counters each, CS guarantees an error bound of at most $\sqrt{\frac{e \cdot \sum_y (f(y))^2}{w}}$ with a confidence of $1-e^{-d}$, i.e., $\Pr\left(|\hat{f}(x) - f(x)| \leq \sqrt{\frac{e \cdot \sum_y (f(y))^2}{w}}\right) \geq 1-e^{-d}$ for any key x. The error term above is smaller than CMS's error term when the frequency distribution is less skewed, as the root slows its growth to $O(\sqrt{N})$ (vs. N) when the summands are small. The quantity $\sqrt{\sum_y (f(y))^2}$ is known as the ℓ_2-norm of the key counts in the stream, which we denote by N_2.

Accommodating Skew. Similarly to CMS, CS wastes memory under skewed workloads due to using uniformly-sized counters. Sketchbook$_{\text{CS}}$ addresses this problem by generalizing CS to use a tailored version of VALE. To this end, it appends a sign bit to each stub. This allows it to store negative counter values by setting the corresponding sign bit to 1 and using VALE as before to encode the counter's absolute value.

Accommodating Unbounded Growth. As with CMS, using CS to process a growing stream yields estimation errors that grow with the total key count N. To resolve this problem, Sketchbook$_{\text{CS}}$ extends CS to expand the number of its counters. To this end, it creates a chain of sketches, similar to that of Sketchbook$_{\text{CMS}}$. It applies the same hashing scheme with the use of mother hashes, but also makes use of shared $\sigma_1, \ldots, \sigma_d$ to derive consistent update directions across the sketches. It applies the same expansion and contraction procedures, and handles a query by summing the estimates of the corresponding arrays and taking the median. It also speeds up queries by aggregating the old sketches into the last one.

Sketchbook$_{\text{CS}}$ differs from Sketchbook$_{\text{CMS}}$ in one key aspect: the estimation error of a sketch in its chain with arrays of size w scales with the stream's ℓ_2-norm, i.e., as $\sqrt{N_2^2/w}$, as opposed to N/w. The value of N_2 varies based on the workload, with uniform workloads having $N_2 \approx \sqrt{N}$ and skewed workloads having $N_2 \approx N$. Sketchbook$_{\text{CS}}$ must therefore expand based on N_2^2 to minimize its memory footprint while ensuring bounded errors for any workload. To this end, it defines its size function as $W(N_2^2)$. Whenever $W(N_2^2)$ doubles, it expands, and whenever it halves, it contracts.

To compute $W(N_2^2)$, Sketchbook$_{\text{CS}}$ must track N_2. It does so by employing an array of AMS sketches to approximate N_2 [2]. An AMS sketch is a specialized CS with a single array consisting of a single counter. Each insertion increments the counter along its update direction and a deletion decrements it. The following theorem holds for AMS sketches:

Theorem 4.1 (Theorem 2.2 of [2], specialized). *One can approximate N_2^2 within a factor of two with a probability of $1 - p$ using $4 \cdot \log 1/p$ AMS sketches. Estimation proceeds by squaring the sketches, averaging them in groups of 4, and taking the median.*

Thus, it suffices to keep $O(\log N)$ AMS sketches in memory to approximate N_2 with high probability at all points in time. Since N is bounded by the maximum value representable in a 64-bit machine word in practice, Sketchbook$_{\text{CS}}$ employs 64 AMS sketches. It uses SIMD to quickly update[9] and query these sketches.

Similarly to Sketchbook$_{\text{CMS}}$, using a sub-linear power size function of the form $W(N_2^2) = N_2^{2\alpha}/\epsilon$ in Sketchbook$_{\text{CS}}$ yields an error bound of $O(N_2^{1-\alpha})$, matching that of a preallocated CS of size $W(N_2^2)$, which is optimal [8]. Moreover, setting $W(N_2^2) = N_2^2/\epsilon$ yields an error bound of $O(\epsilon \cdot \log N_2)$ with the least possible memory.

5 THEORETICAL ANALYSIS

We prove Sketchbook's accuracy, confidence, and performance guarantees for general streams of keys. We further show that Sketchbook is space-optimal by proving a memory lower bound for frequency estimation sketches that do not know the stream's ultimate length.

Accuracy and Confidence.

Memory Footprint. Since VALE's tuning procedure ensures few bits are unused per counter and few chunks use tails arrays, Sketchbook's memory footprint is dominated by the space occupied by the stubs and the extensions. For a single counter, this cost is:

Lemma 5.1. *When working with unsigned counters and stubs of length s, VALE encodes a counter value v in s bits if $v < 2^s$ and in $4 + 1.26 \cdot \log_2 v$ bits otherwise. Signed counters take one more bit to encode the sign in both cases.*

Proof. If $v < 2^s$, it fits in its stub and occupies s bits. Otherwise, its s lower-order bits are stored in its stub, and its higher-order bits encoding $\lfloor v/2^s \rfloor$ are stored in an extension. This extension

[9] More generally, it is possible to maintain k AMS sketches, where k is the machine word size, in constant time with high probability to approximate N_2. We omit the proof of this statement due to space constraints.

has $\lfloor \log_3 \lfloor v/2^s \rfloor \rfloor + 2$ fragments to represent the digits and the delimiter. Thus, it occupies

$$2 \cdot (\lfloor \log_3 \lfloor v/2^s \rfloor \rfloor + 2) \leq 2 \cdot (\log_3 2 \cdot \log_2 \lfloor v/2^s \rfloor + 2) \leq 4 - s + 1.26 \cdot \log_2 v$$

bits. This, combined with the cost of the stub, yields the result. □

Applying Lemma 5.1 to all counters yields:

Theorem 5.2. *When an array of w counters has processed at total of N increments and decrements, its total memory footprint is $O(w \cdot (\log_2(N/w) + 1))$ bits. Applied to all d arrays in Sketchbook when sketches are not aggregated, this corresponds to a memory footprint of $O(d \cdot W(N) \cdot (\log_2(N/W(N)) + 1))$ bits.*

Proof. Denoting the signed value of the i-th counter by v_i and applying Lemma 5.1, the array occupies $m = \sum_i O(1 + \log_2|v_i|) = O\left(w + \sum_i \log_2|v_i|\right)$ bits. Due to the concavity of the logarithm, it must be that $\sum_i \log_2|v_i| \leq w \cdot \log_2(1/w \cdot \sum_i |v_i|)$. Because the logarithm is an increasing function and $\sum_i |v_i| \leq N$, it must be that $w \cdot \log_2(1/w \cdot \sum_i |v_i|) \leq w \cdot \log_2(N/w)$. Plugging this into the first expression in the proof yields $m \leq O(w \cdot (\log_2(N/w) + 1))$. Recalling that the last sketch in Sketchbook's chain dominates the memory and that its arrays are of size $\approx W(N)$ proves the result. □

When $W(N)$ is a sub-linear power, the bound for Sketchbook in Theorem 5.2 still holds when aggregating the sketches, since the total increments and decrements in the last sketch is $O(N)$. Yet, when $W(N) = \widetilde{O}(N)$, the total increments and decrements in the last sketch is $O(N \log N)$. As such, the space bound increases slightly by an additive $O(d \cdot W(N) \cdot \log\log N)$ bits.

Memory Footprint Lower Bound. To prove a memory footprint lower bound for frequency estimation sketches, we use them to implement filters and leverage their memory lower bounds. A filter is a compact data structure that answers whether a query key exists in a given set of keys. A filter never returns a false negative, but it may return a false positive with a probability called the *False Positive Rate (FPR)*. The lower the FPR, the larger the filter's memory footprint must be. The following theorem states a memory footprint lower bound for filters supporting sets of unknown size:

Theorem 5.3 (Theorem 3.1 of [45], Simplified). *Letting n denote the size of a growing set of keys, any filter that maintains an FPR of at most δ for this set at all times must have a memory footprint of at least $n \cdot (\log_2 1/\delta + \log_2 \log_2 n) - \Theta(n)$ bits.*

Therefore, a frequency estimation sketch that can be used to implement such a filter must also use at least this much memory.

To illustrate the argument, consider a frequency estimation sketch with only overestimation errors (e.g., CMS). Suppose this sketch guarantees an error of at most $E(N)$ and a confidence of $1-\delta$ for streams of length N. One can use this sketch to implement a filter with an FPR of δ over a set of size $N/(E(N)+1)$ by inserting each key $E(N)+1$ times. Filtering proceeds by checking if the frequency of a given key is at least $E(N)+1$, returning a positive if it is and a negative otherwise. Querying a key within the set returns an estimate that is always at least as large as $E(N)+1$ (i.e., a true positive) while querying a key outside of the set only yields an estimate of $E(N)+1$ and above (i.e., a false positive) with a probability of δ. As such, Theorem 5.3 must apply to this sketch:

Theorem 5.4. *Any frequency estimation sketch with only overestimation errors that processes growing streams with an error bound of $E(N)$ and a confidence of $1-\delta$ must have a memory footprint of at least $\Omega(N/E(N) \cdot [\log 1/\delta + \log\log(N/E(N))])$ bits.*

It is difficult to implement a filter with a sketch that has two-sided errors, since underestimations may introduce false negatives. Nevertheless, we prove in Appendix B that if the estimates of such a sketch have a variance that yields an error bound of $E(N)$ with a confidence of $1-\delta$ (similarly to CS), the same lower bound holds:

Theorem 5.5. *A linear frequency estimation sketch providing an estimate with a variance of $E(N)/\delta$ for any key when processing a stream of size N, uses $\Omega\left(N/E(N) \cdot [\log 1/\delta + \log\log(N/E(N))]\right)$ bits of memory.*

Recall from Theorem 5.2 that using a constant number of arrays d for a constant confidence bound leads to a memory footprint of $O(W(N) \cdot (\log_2(N/W(N)+1))$ bits for Sketchbook. One can verify that when $W(N)$ is a sub-linear power or a linear function, this memory footprint can be expressed as $O\left(N/E(N) \cdot \log(N/E(N))\right)$ bits, where $E(N)$ is the error corresponding to the choice of $W(N)$. This almost matches the above memory lower bounds with a slight difference in the logarithmic terms. Compressing the counter arrays by coarsely quantizing their counter values and applying run-length encoding allows Sketchbook to match these lower bounds.

Performance. As Sketchbook handles insertions, deletions, and queries using the last sketch in the chain, their cost is the same as the insertion or deletion cost of the last sketch, i.e., $O(d)$ for Sketchbook$_{\text{CMS}}$. Moreover, the amortized cost of expansions and contractions is constant, as they modify $d \cdot W(N) \leq N$ counters every time N more than doubles or halves.

6 EVALUATION

Platform. We run experiments on a Fedora 39 machine with an Intel Xeon w7-2495X processor (4.8 GHz) with 24 cores and 48 hyper-threads. Our machine has a 1920 KB L1, a 48 MB L2, and a 45 MB L3 cache, along with 64 GBs of main memory.

7 RELATED WORK

Additionally, both counter merging and counter sharing non-trivially alter the statistical behavior of the counter values, complicating the integration of other orthogonal techniques like Bayesian inference for improving accuracy [15, 17] and convolution for join size estimation [25].

Mention Spectral Bloom Filters as a related structure to CMS.

Mention the work on analyzing sketches for the zipfian distribution.

Say something about the tuner papers [42] (+ RA-Sketch, if it's out).

8 CONCLUSION

REFERENCES

[1] Amirali Aghazadeh, Ryan Spring, Daniel Lejeune, Gautam Dasarathy, Anshumali Shrivastava, and richard baraniuk. 2018. MISSION: Ultra Large-Scale Feature Selection using Count-Sketches. In *Proceedings of the 35th International Conference on Machine Learning (Proceedings of Machine Learning Research)*, Jennifer Dy and Andreas Krause (Eds.), Vol. 80. PMLR, 80–88. https://proceedings.mlr.press/v80/aghazadeh18a.html

[2] Noga Alon, Yossi Matias, and Mario Szegedy. 1999. The space complexity of approximating the frequency moments. *J. Comput. System Sci.* 58, 1 (1999), 137–147. https://doi.org/10.1006/jcss.1998.1627

[3] Daniel Anderson, Pryce Bevan, Kevin Lang, Edo Liberty, Lee Rhodes, and Justin Thaler. 2017. A high-performance algorithm for identifying frequent items in data streams. In *Proceedings of the 2017 Internet Measurement Conference* (London, United Kingdom) *(IMC '17)*. Association for Computing Machinery, New York, NY, USA, 268–282. https://doi.org/10.1145/3131365.3131407

[4] Ran Basat, Gil Einziger, Michael Mitzenmacher, and Shay Vargaftik. 2021. SALSA: Self-Adjusting Lean Streaming Analytics. 864–875. https://doi.org/10.1109/ICDE51399.2021.00080

[5] F. Bostanci, Ismail Yüksel, Ataberk Olgun, Konstantinos Kanellopoulos, Yahya Tuğrul, Abdullah Giray Yaglikçi, Mohammad Sadrosadati, and Onur Mutlu. 2024. CoMeT: Count-Min-Sketch-based Row Tracking to Mitigate RowHammer at Low Cost. 593–612. https://doi.org/10.1109/HPCA57654.2024.00050

[6] Vladimir Braverman, Stephen R. Chestnut, David P. Woodruff, and Lin F. Yang. 2016. Streaming Space Complexity of Nearly All Functions of One Variable on Frequency Vectors. In *Proceedings of the 35th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems* (San Francisco, California, USA) *(PODS '16)*. Association for Computing Machinery, New York, NY, USA, 261–276. https://doi.org/10.1145/2902251.2902282

[7] Lu Cao, Qilong Shi, Yuxi Liu, Hanyue Zheng, Yao Xin, Wenjun Li, Tong Yang, Yangyang Wang, Yang Xu, Weizhe Zhang, and Mingwei Xu. 2024. Bubble Sketch: A High-performance and Memory-efficient Sketch for Finding Top-Items in Data Streams. In *CIKM*. 3653–3657. https://doi.org/10.1145/3627673.3679882

[8] Moses Charikar, Kevin Chen, and Martin Farach-Colton. 2004. Finding frequent items in data streams. *Theoretical Computer Science* 312, 1 (2004), 3–15. https://doi.org/10.1016/S0304-3975(03)00400-6 Automata, Languages and Programming.

[9] Min Chen, Shigang Chen, and Zhiping Cai. 2017. Counter Tree: A Scalable Counter Architecture for Per-Flow Traffic Measurement. *IEEE/ACM Trans. Netw.* 25, 2 (April 2017), 1249–1262. https://doi.org/10.1109/TNET.2016.2621159

[10] Qizhi Chen, Yisen Hong, Yuhan Wu, Tong Yang, and Bin Cui. 2024. CodingSketch: A Hierarchical Sketch with Efficient Encoding and Recursive Decoding. In *ICDE*. 1592–1605. https://doi.org/10.1109/ICDE60146.2024.00130

[11] David Clark. 1997. *Compact PAT trees.* Ph.D. Dissertation. http://hdl.handle.net/10012/64

[12] Felix Cloutier. 2023. *PDEP — Parallel Bits Deposit.* https://www.felixcloutier.com/x86/pdep

[13] Graham Cormode and S. Muthukrishnan. 2005. An improved data stream summary: the count-min sketch and its applications. *J. Algorithms* 55, 1 (April 2005), 58–75. https://doi.org/10.1016/j.jalgor.2003.12.001

[14] G. Cormode and S. Muthukrishnan. 2005. What's new: finding significant differences in network data streams. *IEEE/ACM Transactions on Networking* 13, 6 (2005), 1219–1232. https://doi.org/10.1109/TNET.2005.860096

[15] Francesco Da Dalt and Adrian Perrig. 2025. BFES: Towards Optimal Bayesian Frequency Estimation Sketches in Data-Streams. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. 2337–2350. https://doi.org/10.1109/ICDE65448.2025.00177

[16] Zhenwei Dai, Aditya Desai, Reinhard Heckel, and Anshumali Shrivastava. 2021. Active Sampling Count Sketch (ASCS) for Online Sparse Estimation of a Trillion Scale Covariance Matrix. In *Proceedings of the 2021 International Conference on Management of Data* (Virtual Event, China) *(SIGMOD '21)*. Association for Computing Machinery, New York, NY, USA, 352–364. https://doi.org/10.1145/3448016.3457327

[17] Francesco Da Dalt, Simon Scherrer, and Adrian Perrig. 2022. Bayesian Sketches for Volume Estimation in Data Streams. *Proc. VLDB Endow.* 16, 4 (Dec. 2022), 657–669. https://doi.org/10.14778/3574245.3574252

[18] Niv Dayan, Ioana Bercea, and Rasmus Pagh. 2024. Aleph Filter: To Infinity in Constant Time. arXiv:2404.04703 [cs.DB] https://arxiv.org/abs/2404.04703

[19] Niv Dayan, Ioana Bercea, Pedro Reviriego, and Rasmus Pagh. 2023. InfiniFilter: Expanding Filters to Infinity and Beyond. *Proc. ACM Manag. Data* 1, 2, Article 140 (jun 2023), 27 pages. https://doi.org/10.1145/3589285

[20] Fan Deng and Davood Rafiei. [n.d.]. New Estimation Algorithms for Streaming Data: Count-min Can Do More. https://webdocs.cs.ualberta.ca/~drafiei/papers/cmm.pdf

[21] Rui Ding, Shibo Yang, Xiang Chen, and Qun Huang. 2023. BitSense: Universal and Nearly Zero-Error Optimization for Sketch Counters with Compressive Sensing. In *Proceedings of the ACM SIGCOMM 2023 Conference* (New York, NY, USA) *(ACM SIGCOMM '23)*. Association for Computing Machinery, New York, NY, USA, 220–238. https://doi.org/10.1145/3603269.3604865

[22] Guoju Gao, Zhaorong Qian, He Huang, and Yang Du. 2023. An Adaptive Counter-Splicing-Based Sketch for Efficient Per-Flow Size Measurement. 1–4. https://doi.org/10.1109/IWQoS57198.2023.10188733

[23] Lukasz Golab. 2009. *Stream Models.* Springer US, Boston, MA, 2834–2836. https://doi.org/10.1007/978-0-387-39940-9_370

[24] Mayank Goswami, Rasmus Pagh, Francesco Silvestri, and Johan Sivertsen. 2017. Distance sensitive bloom filters without false negatives. In *Proceedings of the Twenty-Eighth Annual ACM-SIAM Symposium on Discrete Algorithms* (Barcelona, Spain) *(SODA '17)*. Society for Industrial and Applied Mathematics, USA, 257–269.

[25] Mike Heddes, Igor Nunes, Tony Givargis, and Alex Nicolau. 2024. Convolution and Cross-Correlation of Count Sketches Enables Fast Cardinality Estimation of Multi-Join Queries. *Proc. ACM Manag. Data* 2, 3, Article 129 (May 2024), 26 pages. https://doi.org/10.1145/3654932

[26] Qun Huang, Xin Jin, Patrick P. C. Lee, Runhui Li, Lu Tang, Yi-Chao Chen, and Gong Zhang. 2017. SketchVisor: Robust Network Measurement for Software Packet Processing. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication* (Los Angeles, CA, USA) *(SIGCOMM '17)*. Association for Computing Machinery, New York, NY, USA, 113–126. https://doi.org/10.1145/3098822.3098831

[27] Yesdaulet Izenov. 2023. *Query Optimization using Sketches in Relational Database Systems.* PhD thesis. University of California, Merced, Merced, CA. Available at https://escholarship.org/uc/item/5k8334fs.

[28] Yesdaulet Izenov, Asoke Datta, Florin Rusu, and Jun Hyung Shin. 2021. COMPASS: Online Sketch-based Query Optimization for In-Memory Databases. In *Proceedings of the 2021 International Conference on Management of Data* (Virtual Event, China) *(SIGMOD '21)*. Association for Computing Machinery, New York, NY, USA, 804–816. https://doi.org/10.1145/3448016.3452840

[29] Peng Jia, Pinghui Wang, Junzhou Zhao, Shuo Zhang, Yiyan Qi, Min Hu, Chao Deng, and Xiaohong Guan. 2021. Bidirectionally Densifying LSH Sketches with Empty Bins. In *Proceedings of the 2021 International Conference on Management of Data* (Virtual Event, China) *(SIGMOD '21)*. Association for Computing Machinery, New York, NY, USA, 830–842. https://doi.org/10.1145/3448016.3452833

[30] Balachander Krishnamurthy, Subhabrata Sen, Yin Zhang, and Yan Chen. 2003. Sketch-based change detection: methods, evaluation, and applications. In *Proceedings of the 3rd ACM SIGCOMM Conference on Internet Measurement* (Miami Beach, FL, USA) *(IMC '03)*. Association for Computing Machinery, New York, NY, USA, 234–247. https://doi.org/10.1145/948205.948236

[31] Florian Kurpicz. 2022. Engineering Compact Data Structures for Rank and Select Queries on Bit Vectors. In *String Processing and Information Retrieval – 29th International Symposium, SPIRE 2022, Concepción, Chile, November 8–10, 2022, Proceedings. Ed.: D. Arroyuelo (Lecture Notes in Computer Science)*, Vol. 13617. Springer International Publishing, 257–272. https://doi.org/10.1007/978-3-031-20643-6_19

[32] Haoyu Li, Qizhi Chen, Yixin Zhang, Tong Yang, and Bin Cui. 2022. Stingy sketch: a sketch framework for accurate and fast frequency estimation. *Proc. VLDB Endow.* 15, 7 (March 2022), 1426–1438. https://doi.org/10.14778/3523210.3523220

[33] Jizhou Li, Zikun Li, Yifei Xu, Shiqi Jiang, Tong Yang, Bin Cui, Yafei Dai, and Gong Zhang. 2020. WavingSketch: An Unbiased and Generic Sketch for Finding Top-k Items in Data Streams. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (Virtual Event, CA, USA) *(KDD '20)*. Association for Computing Machinery, New York, NY, USA, 1574–1584. https://doi.org/10.1145/3394486.3403208

[34] Shangsen Li, Lailong Luo, Deke Guo, Qianzhen Zhang, and Pengtao Fu. 2020. A survey of sketches in traffic measurement: Design, Optimization, Application and Implementation. https://api.semanticscholar.org/CorpusID:236140679

[35] Shangsen Li, Lailong Luo, Deke Guo, Qianzhen Zhang, and Pengtao Fu. 2020. A survey of sketches in traffic measurement: Design, Optimization, Application and Implementation. https://api.semanticscholar.org/CorpusID:236140679

[36] Weihe Li and Paul Patras. 2023. P-Sketch: A Fast and Accurate Sketch for Persistent Item Lookup. *IEEE/ACM Trans. Netw.* 32, 2 (Aug. 2023), 987–1002. https://doi.org/10.1109/TNET.2023.3306897

[37] Weihe Li and Paul Patras. 2023. Tight-Sketch: A High-Performance Sketch for Heavy Item-Oriented Data Stream Mining with Limited Memory Size. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management* (Birmingham, United Kingdom) *(CIKM '23)*. Association for Computing Machinery, New York, NY, USA, 1328–1337. https://doi.org/10.1145/3583780.3615080

[38] Zaoxing Liu, Antonis Manousis, Gregory Vorsanger, Vyas Sekar, and Vladimir Braverman. 2016. One Sketch to Rule Them All: Rethinking Network Flow Monitoring with UnivMon. In *Proceedings of the 2016 ACM SIGCOMM Conference* (Florianopolis, Brazil) *(SIGCOMM '16)*. Association for Computing Machinery, New York, NY, USA, 101–114. https://doi.org/10.1145/2934872.2934906

[39] Zirui Liu, Yixin Zhang, Yifan Zhu, Ruwen Zhang, Tong Yang, Kun Xie, Sha Wang, Tao Li, and Bin Cui. 2023. TreeSensing: Linearly Compressing Sketches with Flexibility. *Proc. ACM Manag. Data* 1, 1, Article 56 (May 2023), 28 pages. https://doi.org/10.1145/3588910

[40] Dzejla Medjedovic, Emin Tahirovic, and Ines Dedovic. 2025. *Algorithms and data structures for massive datasets.* Manning Publications Co., Chapter 4.

[41] Ahmed Metwally, Divyakant Agrawal, and Amr El Abbadi. 2005. Efficient computation of frequent and top-k elements in data streams. In *Proceedings of the 10th International Conference on Database Theory* (Edinburgh, UK) *(ICDT'05)*. Springer-Verlag, Berlin, Heidelberg, 398–412. https://doi.org/10.1007/978-3-540-30570-5_27

[42] Ruijie Miao, Fenghao Dong, Yikai Zhao, Yiming Zhao, Yuhan Wu, Kaicheng Yang, Tong Yang, and Bin Cui. 2023. SketchConf: A Framework for Automatic Sketch

Configuration. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. 2022–2035. https://doi.org/10.1109/ICDE55515.2023.00157

[43] Jayadev Misra and David Gries. 1982. *Finding Repeated Elements*. Technical Report. USA.

[44] Daisuke Okanohara and Kunihiko Sadakane. 2007. Practical entropy-compressed rank/select dictionary. In *Proceedings of the Meeting on Algorithm Engineering & Expermiments* (New Orleans, Louisiana). Society for Industrial and Applied Mathematics, USA, 60–70.

[45] Rasmus Pagh, Gil Segev, and Udi Wieder. 2013. How to Approximate a Set without Knowing Its Size in Advance. *2013 IEEE 54th Annual Symposium on Foundations of Computer Science* (2013), 80–89. https://api.semanticscholar.org/CorpusID:10365891

[46] Prashant Pandey, Michael A. Bender, Rob Johnson, and Rob Patro. 2017. A General-Purpose Counting Filter: Making Every Bit Count. In *Proceedings of the 2017 ACM International Conference on Management of Data* (Chicago, Illinois, USA) *(SIGMOD '17)*. Association for Computing Machinery, New York, NY, USA, 775–787. https://doi.org/10.1145/3035918.3035963

[47] Pratanu Roy, Arijit Khan, and Gustavo Alonso. 2016. Augmented Sketch: Faster and More Accurate Stream Processing. In *Proceedings of the 2016 International Conference on Management of Data* (San Francisco, California, USA) *(SIGMOD '16)*. Association for Computing Machinery, New York, NY, USA, 1449–1463. https://doi.org/10.1145/2882903.2882948

[48] Qilong Shi, Chengjun Jia, Wenjun Li, Zaoxing Liu, Tong Yang, Jianan Ji, Gaogang Xie, Weizhe Zhang, and Minlan Yu. 2024. BitMatcher: Bit-level Counter Adjustment for Sketches. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. 4815–4827. https://doi.org/10.1109/ICDE60146.2024.00366

[49] Yoshihiro Shibuya, Djamal Belazzougui, and Gregory Kucherov. 2021. Set-Min sketch: a probabilistic map for power-law distributions with application to k-mer annotation. *bioRxiv* (2021). https://doi.org/10.1101/2020.11.14.382713 arXiv:https://www.biorxiv.org/content/early/2021/02/25/2020.11.14.382713.full.pdf

[50] Lu Tang, Qun Huang, and Patrick P. C. Lee. 2019. MV-Sketch: A Fast and Compact Invertible Sketch for Heavy Flow Detection in Network Data Streams. In *IEEE INFOCOM 2019 - IEEE Conference on Computer Communications* (Paris, France). IEEE Press, 2026–2034. https://doi.org/10.1109/INFOCOM.2019.8737499

[51] Daniel Ting. 2018. Data Sketches for Disaggregated Subset Sum and Frequent Item Estimation. In *Proceedings of the 2018 International Conference on Management of Data* (Houston, TX, USA) *(SIGMOD '18)*. Association for Computing Machinery, New York, NY, USA, 1129–1140. https://doi.org/10.1145/3183713.3183759

[52] Pinghui Wang, Yiyan Qi, Yuanming Zhang, Qiaozhu Zhai, Chenxu Wang, John C.S. Lui, and Xiaohong Guan. 2019. A Memory-Efficient Sketch Method for Estimating High Similarities in Streaming Sets. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (Anchorage, AK, USA) *(KDD '19)*. Association for Computing Machinery, New York, NY, USA, 25–33. https://doi.org/10.1145/3292500.3330825

[53] Alexander Wei. 2022. Optimal Las Vegas Approximate Near Neighbors in ℓ_p. *ACM Trans. Algorithms* 18, 1, Article 7 (Jan. 2022), 27 pages. https://doi.org/10.1145/3461777

[54] Yuhan Wu, Shiqi Jiang, Siyuan Dong, Zheng Zhong, Jiale Chen, Yutong Hu, Tong Yang, Steve Uhlig, and Bin Cui. 2023. MicroscopeSketch: Accurate Sliding Estimation Using Adaptive Zooming. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Long Beach, CA, USA) *(KDD '23)*. Association for Computing Machinery, New York, NY, USA, 2660–2671. https://doi.org/10.1145/3580305.3599432

[55] Tong Yang, Junzhi Gong, Haowei Zhang, Lei Zou, Lei Shi, and Xiaoming Li. 2018. HeavyGuardian: Separate and Guard Hot Items in Data Streams. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (London, United Kingdom) *(KDD '18)*. Association for Computing Machinery, New York, NY, USA, 2584–2593. https://doi.org/10.1145/3219819.3219978

[56] Tong Yang, Jie Jiang, Peng Liu, Qun Huang, Junzhi Gong, Yang Zhou, Rui Miao, Xiaoming Li, and Steve Uhlig. 2018. Elastic sketch: adaptive and fast network-wide measurements. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication* (Budapest, Hungary) *(SIGCOMM '18)*. Association for Computing Machinery, New York, NY, USA, 561–575. https://doi.org/10.1145/3230543.3230544

[57] Tong Yang, Haowei Zhang, Jinyang Li, Junzhi Gong, Steve Uhlig, Shigang Chen, and Xiaoming Li. 2019. HeavyKeeper: An Accurate Algorithm for Finding Top-k Elephant Flows. *IEEE/ACM Trans. Netw.* 27, 5 (Oct. 2019), 1845–1858. https://doi.org/10.1109/TNET.2019.2933868

[58] Tong Yang, Yang Zhou, Hao Jin, Shigang Chen, and Xiaoming Li. 2017. Pyramid sketch: a sketch framework for frequency estimation of data streams. *Proc. VLDB Endow.* 10, 11 (Aug. 2017), 1442–1453. https://doi.org/10.14778/3137628.3137652

[59] Boyu Zhang, He Huang, Yu-E. Sun, Yang Du, and Dan Wang. 2024. JigsawSketch: a fast and accurate algorithm for finding top-k elephant flows in high-speed networks. *Science China Information Sciences* 67, 4 (20 Mar 2024), 142101. https://doi.org/10.1007/s11432-022-3794-1

[60] Qingpeng Zhang, Jason Pell, Rosangela Canino-Koning, Adina Chuang Howe, and C Titus Brown. 2014. These are not the k-mers you are looking for: efficient online k-mer counting using a probabilistic data structure. *PloS one* 9, 7 (2014), e101271.

[61] Dong Zhou, David G. Andersen, and Michael Kaminsky. 2013. Space-Efficient, High-Performance Rank and Select Structures on Uncompressed Bit Sequences. In *Experimental Algorithms*, Vincenzo Bonifaci, Camil Demetrescu, and Alberto Marchetti-Spaccamela (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 151–163.

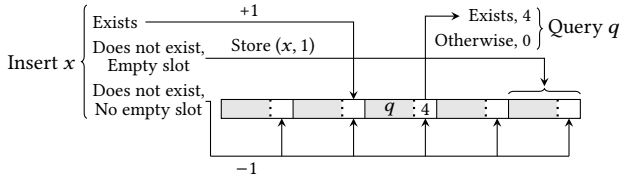


Figure 11: MG uses dedicated counters to track frequent keys.

A APPLYING SKETCHBOOK TO MISRA-GRIES

Misra-Gries (MG). Although CMS and CS benefit from more skew due to distinct keys colliding less frequently, using dedicated counters to track the most frequent keys yields higher accuracy for the same memory compared to hashing them into a counter array. MG applies this insight to provide more accurate estimates under skewed workloads. The third row of Table 2 summarizes the properties of MG and compares it to CMS and CS.

Figure 11 presents an overview of MG. MG maintains an array of slots, each storing a key (shown in gray) and a dedicated frequency counter (shown in white). MG estimates the frequency of each key in the array as the value of its counter. It returns an estimate of zero for the untracked keys. When inserting a key x, MG first checks if it has a dedicated counter in some slot. If it does, MG increments x's counter. If not, and the array has an empty slot, MG stores x in it with a count of 1. If x is not in the array and the array is full, MG decrements all counters and discards any keys whose counters become zero. Such keys are unlikely to be among the most frequent and their slots are better used for tracking more frequent keys. Note that x is not stored in the newly emptied slots and is only stored if it is inserted when some slot is already empty. MG does not support deletions without deteriorating the error bound [3]. This is because it lacks information about the untracked keys, which may become the heavy hitters after the current heavy hitters are deleted, thereby failing to place them in its array.

Having more slots in the array allows MG to track more keys without discarding them, leading to more accurate estimates. Since MG is a deterministic data structure that does not use hashing, unlike CMS and CS, it does not suffer from hash collisions, and its estimations have full confidence. Using an array of w slots, MG estimates the frequency of any key with an error of at most $\frac{N}{w}$, i.e., $|\hat{f}(x) - f(x)| \leq \frac{N}{w}$ for any key x. Since it uses dedicated counters for the heavy hitters, it also provides a strong "*Tail Guarantee*," stating that the error for any key is at most $\min_{1 \leq i \leq w} \frac{N^{\text{res}(i)}}{w-i+1}$, where $N^{\text{res}(i)}$ is the total frequency of all keys not among the top i. Note that $N^{\text{res}(i)}$ is much smaller than N when the data is skewed.

Since MG only increments a key's counter if it is in the array and decrements counters otherwise, it only underestimates frequencies. Thus, its estimates are biased. For similar reasons to CMS, this bias cannot be removed by enlarging estimates. Yet, probabilistically replacing the array's least frequent keys can yield unbiased estimates [51]. Other variations of MG achieve higher accuracy in practice, but suffer from two-sided errors in exchange [41].

Typical implementations of MG use a hash table to implement the array of keys, yielding constant-time queries. They also optimize insertions by lazily decrementing the counters and employing an auxiliary heap to locate and discard keys with zero counts. The query time can be further improved to $O(1)$ using a specialized data structures similar to a linked list in exchange for a higher memory footprint [41].

Accommodating Skew. The counts stored in MG may still exhibit skew within themselves. This causes memory wastage if uniformly-sized counters are used. Sketchbook$_{\text{MG}}$ addresses this issue by storing the counter values in a separate array that mirrors the ordering of the keys within the array of slots. It applies VALEto space efficiently encode these counter values. No modification is required to VALE, as the counter values are all positive.

Accommodating Skew. To bound the error for growing streams, Sketchbook$_{\text{MG}}$ generalizes MG to expand its array of slots. Since keys are not hashed into the array of slots and can be placed in any slot, Sketchbook$_{\text{MG}}$ expands this array on a slot-by-slot basis, unlike Sketchbook$_{\text{CMS}}$ and Sketchbook$_{\text{CS}}$ that double the size of their arrays.

B OMITTED PROOFS

Theorem 5.5. *A linear frequency estimation sketch providing an estimate with a variance of $E(N)/\delta$ for any key when processing a stream of size N, uses $\Omega\left(N/E(N) \cdot [\log 1/\delta + \log\log(N/E(N))]\right)$ bits of memory.*

Proof. It is impossible to implement a filter using a sketch with two-sided errors by thresholding the estimates. This is because underestimation errors may introduce false negatives. Nevertheless, we show taht the lower bound still holds by adapting the techniques used in the proof of Theorem 3.1 in [45] to these sketches. At a high level, our proof follows a contradiction argument. We show that if a frequency estimation sketch $\mathcal{S}$ with an error bound of $\epsilon(N)$ and confidence of $1-\delta$ exists that is "too space-efficient," it can be used to compress many sequences of $n = N/(\epsilon(N)+1)$ keys $\sigma = \sigma_1, \ldots, \sigma_n$ coming from a large universe U of size $u \gg N$ to a size smaller than their entropy, which must be impossible. We do this by simulating a filter as best as possible with the sketch, despite there being false negatives. Our argument holds regardless of the time complexity of $\mathcal{S}$'s operations.

First, we show that, w.l.o.g., we can assume that $\mathcal{S}$ is a deterministic sketch which only has estimation errors larger than $\epsilon(N)$ on a 2δ fraction of the elements in U. We denote the fraction of such elements in U after inserting each of the sequence σ's keys $\epsilon(N)+1$ times into $\mathcal{S}$ as $\mu_{\mathcal{S}}(\sigma)$. We follow a similar argument to that of Section 3.1 in [45]:

Lemma B.1. *We can fix the internal randomness of the sketch $\mathcal{S}$ such that, for at least half of the possible sequences σ, $\mu_{\mathcal{S}}(\sigma) \leq 2\delta$.*

Proof. The randomized version of $\mathcal{S}$ has a confidence of $1-\delta$. Thus, denoting the internal random bits of $\mathcal{S}$ by r, for any sequence $\sigma \in U^n$, we have that $\mathbb{E}_r[\mu_{\mathcal{S}}(\sigma)] \leq \delta$. Thus, by Markov's inequality, it holds that

$$\Pr_r[\mu_{\mathcal{S}}(\sigma) \geq 2\delta] \leq \frac{1}{2}.$$

Considering the complement even yields

$$\Pr_r[\mu_{\mathcal{S}}(\sigma) < 2\delta] \geq \frac{1}{2}.$$

Thus, the expected number of sequences σ that satisfy $\mu_{\mathcal{S}}(\sigma) < 2\delta$ is at least $u^n/2$, i.e., half of all possible sequences. An averaging argument implies that there is some choice of $\mathcal{S}$'s internal randomness that satisfies $\mu_{\mathcal{S}}(\sigma) < 2\delta$ for at least half of all sequences. □

Thus, given a randomized sketch, we can convert it into a deterministic sketch $\mathcal{S}$ by fixing the internal randomness to the choice mentioned in Lemma B.1 with the desired property.

We now use $\mathcal{S}$ to compress the $u^n/2$ sequences that □